Legal Document Q&A Assistant

Project Title: Legal Document Q&A Assistant

Problem Statement: Reading legal contracts is time-consuming and confusing for non-lawyers. People often sign agreements without fully understanding key clauses like termination, liability, or payment terms — which can lead to costly mistakes. There is no simple, accessible tool that lets a regular user upload a contract and ask questions about it in plain language.

Overview: This application allows users to upload legal documents (PDFs) and interact with them through a conversational chat interface. It retrieves the most relevant clauses based on the user's question and generates accurate, context-grounded answers. Users can also request plain-English simplifications of complex legal language through follow-up questions.

Tech Stack:

- **LangChain components used:**
  - `PyPDFLoader` — document loading
  - `RecursiveCharacterTextSplitter` — chunking
  - `OpenAIEmbeddings` / `HuggingFaceEmbeddings` — text embeddings
  - `Chroma` — vector store
  - LCEL (LangChain Expression Language) — chain building
  - `ChatPromptTemplate` — prompt management
  - `CohereRerank` / `FlashrankRerank` — reranker to re-score retrieved chunks and pick the most relevant ones
  - `MultiVectorRetriever` — supports querying across multiple uploaded documents at once
- **LangGraph components used:**
  - `StateGraph` — overall agent graph
  - `TypedDict` state schema — managing query, history, retrieved docs, and flags
  - Conditional edges — routing between answer and clarification nodes
  - Human-in-the-loop — agent pauses before answering sensitive clauses and asks user to confirm the right clause

- Node functions — `retrieve`, `rerank`, `web_search`, `human_review`, `retry`, `generate_answer`, `check_clarification`, `summarize_clause`
    - `retrieve` — searches vector store and pulls relevant clauses from uploaded documents
    - `rerank` — re-scores retrieved chunks and selects the most relevant one for the query
    - `web_search` — calls Tavily Search API when no relevant clause is found in the document
    - `human_review` — pauses the graph and asks user to confirm the retrieved clause before generating an answer
    - `retry` — if retrieval returns empty or low-quality results, rephrases the query and re-runs retrieval automatically
    - `generate_answer` — takes retrieved clauses or web results and produces a grounded LLM response
    - `check_clarification` — detects if the user wants a simpler explanation and sets the routing flag
    - `summarize_clause` — rewrites the legal answer in plain, simple English

- **Other tools/APIs:**
    - OpenAI GPT-4o / Groq (LLM)
    - Streamlit (chat UI)
    - Python-dotenv (environment management)
    - **Tavily Search API** — web search tool for fetching real-world legal context (e.g. what a clause type means in law, jurisdiction-specific rules)

Key Features:

1. **PDF Upload & Ingestion** — Users upload any contract or legal document; the system automatically chunks and indexes it into a vector store.
2. **Clause-Based Q&A** — Users ask natural language questions and receive answers grounded strictly in the document's content.
3. **Plain-English Summarization** — Users can request a simpler explanation of any retrieved clause via a follow-up question.
4. **Multi-Turn Conversation Memory** — The agent remembers previous exchanges in the session, enabling contextual follow-up questions.
5. **Source Transparency** — Every answer shows the exact clause(s) retrieved from the document so users can verify the response.

Architecture:

The agent has **5 nodes** connected in a graph. Below is the full flow:

Step 1 — Retrieve Node

- User submits a question
- The system searches the vector store across all uploaded documents
- Result is stored in `retrieved_docs`

Step 2 — Rerank Node

- Re-scores all retrieved chunks and picks the most relevant one
- Prevents low-quality or loosely related clauses from reaching the LLM
- Result overwrites `retrieved_docs` with the top-ranked chunk

Step 3 — Retry Node *(conditional)*

- Checks: are the reranked results good enough?
- ✅ YES → move to Step 4
- ❌ NO → rephrase the query and go back to Step 1 (max 2 retries)

Step 4 — Web Search Node *(conditional)*

- Checks: were relevant clauses found in the document?
- ✅ YES → skip this step, move to Step 5
- ❌ NO → call Tavily Search API to find legal context from the web
- Result is stored in `web_search_results`

Step 5 — Human Review Node *(conditional)*

- Checks: is this a sensitive clause (e.g. liability, termination)?
- ✅ YES → agent pauses, shows the retrieved clause to the user and asks "Is this the clause you meant?"
- ❌ NO → move directly to Step 6

Step 6 — Generate Answer Node

- LLM receives: user question + top clause (or web results) + chat history
- Generates a grounded answer stored in `answer`

Step 7 — Clarification Check Node *(conditional)*

- Checks: is the user asking for a simpler explanation?
- ✅ YES → go to Step 8
- ❌ NO → return `answer` as Final Response

Step 8 — Summarize Clause Node *(only if needed)*

- Rewrites the legal answer in plain, simple English
- Returns simplified text as Final Response

Visual Flow:

```
[User Question]
      │
      ▼
┌──────────────┐
│ RETRIEVE     │ ← searches across all uploaded docs
└──────┬───────┘
      │
      ▼
┌──────────────┐
│ RERANK       │ ← picks the most relevant chunk
└──────┬───────┘
      │
      ▼
┌──────────────────────────────────────────┐
│ Results good enough?                     │
│ YES ──────────────────────────────────────► │──┐
│ NO → [RETRY] rephrase + re-retrieve      │  │
└──────────────────────────────────────────┘  │
      │                          │
      ▼                          │
┌──────────────────────────────────────────┐  │
│ Clause found in document?                │◄─┘
│ YES ──────────────────────────────────────► │──┐
│ NO → [WEB SEARCH] (Tavily)               │  │
└──────────────────────────────────────────┘  │
      │                          │
      ▼                          │
┌──────────────────────────────────────────┐◄─┘
│ Sensitive clause?                        │
│ YES → [HUMAN REVIEW] confirm clause      │──┐
│ NO ───────────────────────────────────────► │  │
└──────────────────────────────────────────┘  │
      │                          │
      ▼                          │
┌──────────────┐◄──────────────────────────┘
│ GENERATE ANSWER │ ← uses clause OR web results
└──────┬───────┘
      │
      ▼
┌──────────────────────────────────────────┐
│ User wants simpler explanation?          │
│ NO ───────────────────────────────────────► Final Response
│ YES → [SUMMARIZE CLAUSE]                 │
```

State Management:

Field	Purpose
`query`	Current user question
`chat_history`	All previous Q&A pairs for multi-turn memory
`retrieved_docs`	Clauses fetched from vector store across all uploaded documents
`reranked_docs`	Top-ranked clause after reranking; passed to generate answer node
`retry_count`	Tracks how many times retrieval has been retried (max 2)
`web_search_results`	Results from Tavily if doc has no answer
`human_confirmed`	`True` once user confirms the retrieved clause is correct
`answer`	Generated response from LLM
`needs_clarification`	`True` if user wants plain-English simplification
`source`	`"document"` or `"web"` — shown in UI for transparency

Expected Output: A Streamlit chat interface where users upload a PDF on the sidebar, then chat with the document. Each response displays the generated answer along with an expandable section showing the source clauses retrieved from the document.

Timeline: 3 weeks

- Week 1 — Setup, document ingestion pipeline, basic RAG chain
- Week 2 — LangGraph agent, conditional routing, conversation memory
- Week 3 — Streamlit UI, testing on real contracts, final polish